

The dashboard

The dashboard shows a SecureGate report in your web browser: the totals, every finding, and how to fix each one. It is **read-only**: it reads the report file and never changes anything. It only ever shows **masked** values.

Run it

After scanning the demo repo:

```
make demo
make scan-demo
make ui
```

`make ui` starts the dashboard and opens it in your browser at `http://127.0.0.1:5000`. To stop it, press **Ctrl+C** in the terminal.

To show another report, run this in the SecureGate folder:

```
.venv\Scripts\securegate ui --report findings.json --open
```

- `--report`: the report to show. The default is `findings.json`.
- `--port`: the port number. The default is 5000; choose another one if 5000 is busy, for example `--port 5050`.
- `--open`: open the dashboard in your browser.
- `--debug`: print error details in the terminal. It is off by default.

The dashboard only answers on **127.0.0.1**, the address that always means "this computer": nobody else on your network can open it. It needs no internet connection: it loads nothing from other websites and runs no scripts.

If the report is missing or broken, the dashboard says so and shows the exact command that creates one. Run it, then reload the page. The report is read again every time a page opens, so a new scan appears after a reload.

The pages

Overview (/). Four cards: all findings, blocked, warnings and ignored; select a card to see those findings. One bar per severity shows how many findings have it. "About this scan" says when and where the scan ran, what was scanned (for example the whole Git history), the Gitleaks version and the policy file.

Findings (/findings). A table with the decision, rule, file and line, masked value and reason. Blocked findings come first. The buttons above the table show one decision at a time; they change the address, for example to `/findings?decision=block`, so a filtered view can be bookmarked or shared. Select a row to open its details.

Finding detail (/findings/<id>). Every field of the finding, plus **How to fix**:

- for a blocked key: revoke it at the provider, create a new one, store it in a secret manager, then remove it from the code;

- for a warning or an ignored finding: why it was not blocked, and what to do if it turns out to be real.

If the same secret was found in several places, every place is listed. An id that is not in the report shows a "Not found" page.

Example: `/findings/dcc7b5bbfe3c` is the Stripe key from the demo's deleted script. It shows `sk_1****562d`, a BLOCK badge and four steps, starting with "Revoke it at the provider. In the Stripe Dashboard, open Developers > API keys and roll this key".

Change colours, fonts and spacing

Every design value lives in one file: `src/securegate/ui/static/css/tokens.css`. Change a value there and reload the page.

To change	Edit these tokens
page, card, border and text colours	<code>--color-neutral-...</code>
the accent: links, bars, the active tab	<code>--color-brand-...</code>
the BLOCK, WARN and IGNORE badges	<code>--color-block-...</code> , <code>--color-warn-...</code> , <code>--color-ignore-...</code>
fonts and text sizes	<code>--font-...</code> and <code>--line-height-...</code>
spacing	<code>--spacing-...</code>
rounded corners	<code>--border-radius-...</code>
text size and page width on big screens and projectors	the block starting <code>@media (min-width: 1600px)</code>

The names follow Fluent 2, so a Figma token such as `colorNeutralBackground1` becomes `--color-neutral-background-1`.

Two tests protect the design:

- `app.css`, the layout file, may only use these tokens. A test fails if a colour, size or spacing value is written anywhere else.
- Text colours must meet WCAG AA contrast (at least 4.5 to 1) against their backgrounds. After changing a colour, run `make check` to see whether it still passes.

A decision is never shown by colour alone: it always has a BLOCK, WARN or IGNORE badge.

A sample report for the designers

```
make sample-report
```

This writes `sample_findings.json` with about 20 findings. They come from a real scan of a demo repo that holds the DemoPay app twice: once as usual, and once more under `billing/`. It needs Git and Gitleaks. To look at it:

```
.venv\Scripts\securegate ui --report sample_findings.json --open
```